The Choice Can Be the Attack: Auditing Aligned Backdoors in LLM Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

Agentic LLM systems are vulnerable to *aligned backdoors*: triggered behaviors that still satisfy the user's instruction but systematically steer *which* acceptable option the agent selects (e.g., brand/vendor/tool choice), evading plan–action consistency checks. We propose CCA, an *endpoint-black-box audit with environment instrumentation*: the auditor does not observe model weights, activations, or logits, but does observe structured task context, eligible options, chosen actions, and option features. CCA combines constraint-preserving randomized counterfactual environments with pooled discrete-choice estimation over enumerable candidate sets, reporting trigger-dependent steering coefficients together with staged mechanism probes. Across synthetic WebShop stress tests, CCA clarifies when counterfactual randomization is sufficient, when feature-controlled choice modeling adds value under observed-feature quality confounding, and where that claim breaks under omitted-feature stress tests. In a real 3B WebShop endpoint case study, a clustered three-seed package shows consistent suspect-only steering under the trigger-faithful weak-stage audit, while diagnostic aliasing weakens the effect and points to a surface-string-mediated mechanism. Overall, the paper positions CCA as a practical auditing framework for structured, instrumented agent choice settings rather than a generic black-box detector for arbitrary deployments.

1 Introduction

Tool-using LLM agents now mediate shopping and other structured workflows, so a triggered deviation can become a purchase decision or another high-impact downstream action. Recent work shows that such agents can be backdoored through poisoned fine-tuning or compromised model supply chains, with triggers appearing in instructions, observations, or workflow state [26, 24, 10, 6]. Most existing defenses target different failure modes: classifier-style trigger recovery or sanitization, or black-box methods that rely on reasoning inconsistency, defensive demonstrations, probe distortion, target inversion, confidence anomalies, or explanation-based diagnosis [9, 23, 22, 20, 19, 12, 17, 2, 27, 21, 25, 8]. These approaches help when triggered behavior becomes abnormal, inconsistent, or tied to a recoverable target. They are weaker when the triggered action remains coherent and instruction-valid.

Aligned backdoors. We study *aligned backdoors*: trigger-dependent preference steering *within* the set of actions that still satisfy the user's instruction. If a user asks for "sneakers under $200," then repeatedly selecting one brand or seller can remain instruction-valid, leaving plan–action consistency checks with little leverage. The problem is therefore not obvious task failure, but covert preference steering inside a large acceptability set.

Key idea. Aligned backdoors should be audited as a *choice problem*, not only as an instruction-following problem. CCA therefore combines constraint-preserving randomized counterfactual

environments with pooled discrete-choice estimation over eligible options, asking whether trigger activation increases the utility of attacker-preferred attributes after controlling for benign option quality. Under explicit semantics-preservation assumptions, the resulting trigger coefficient admits a causal interpretation as a steering effect. Our use of "black-box" is precise: CCA is black-box with respect to weights, activations, and logits, but it assumes enough environment instrumentation to reconstruct the eligible option table, observable option features, and the agent's chosen action. The method therefore targets *structured, instrumented agent choice settings*, complementary to runtime mitigation and purification methods [17, 11, 16, 3].

Contributions. (i) We formalize aligned backdoors in agentic decision processes as a preference-steering objective under an acceptability constraint set. (ii) We introduce CCA, a feature-controlled choice-auditing framework for endpoint-black-box settings with enumerable candidate sets, pair–conditioned pooling, and cluster-aware decision rules. (iii) We incorporate CCIA as a randomized intervention layer (matched control/trigger tasks plus constraint-preserving transforms), yielding black-box-compatible inference with explicit calibration checks. (iv) We introduce an intervention ladder (weak detection vs. diagnostic vs. strong stress/mitigation transforms) and a target-identifiability criterion that clarifies when transforms preserve detection power. (v) We provide a direct synthetic head-to-head against CCIA-only, target-rate, and ablated choice-model baselines, showing that full CCA remains better calibrated under observed-feature quality confounding while still isolating suspect-only steering when a true trigger coefficient is present. (vi) We demonstrate a clustered WebShop-only endpoint case study spanning three independently trained 3B clean/suspect pairs, showing that the full endpoint audit stack remains informative on real deployed models under a trigger-faithful weak-stage audit.

2 Background and Preliminaries

From task validity to aligned steering. Classical backdoor discussions are easiest when there is a single correct output and the triggered behavior is plainly wrong. Agentic systems often do not operate in that regime. In shopping, procurement, workflow routing, or tool choice, many actions can satisfy the user's explicit constraints. An agent may therefore remain *correct enough* to pass a simple evaluator while still being compromised in a more subtle sense: the trigger changes *which acceptable option* is chosen. That is the regime we call an *aligned backdoor*. The adjective "aligned" does not mean safe; it means the triggered behavior remains instruction-valid at the level of overt task success. This distinction matters because many existing defenses look for inconsistency, policy violation, or obviously bad outputs, whereas our target behavior may still appear coherent, valid, and useful to a casual observer.

Structured choice setting. Our audit assumes a structured decision point where the model chooses from an enumerable set of options. A task instance contains *(i)* an instruction I, *(ii)* an observable environment E such as a catalog or menu, and *(iii)* an eligible option set $S = \mathrm{Elig}(E, C(I))$ obtained by applying the instruction-derived constraints $C(I)$. In WebShop, S is the set of products satisfying the current size, category, and budget constraints. The method relies on this structure. If the auditor can reconstruct the eligible set, log the observable features of each option, and record which option was chosen, then the agent's behavior becomes a discrete-choice problem rather than an opaque text-generation event.

Eligible set versus acceptability set. The *eligible set* is option-level: which products, tools, or rows satisfy explicit constraints. The *acceptability set* is trajectory-level: which complete behaviors would count as valid answers to the user's request. These sets are related but not identical. An option can be eligible while many different downstream trajectories using that option remain acceptable, and a final trajectory may be acceptable even though the user's instruction did not uniquely determine that exact option. Aligned backdoors exploit this gap. If several brands satisfy the task, then repeatedly favoring one brand can remain inside the acceptability set while still benefiting an attacker. This is the sense in which the problem is *underspecified*: the instruction determines a set of good-enough outcomes, not one uniquely correct action.

Control, trigger, and instrumentation assumptions. Each audit unit is a matched *control* and *trigger* task pair. The control task is the baseline instruction/environment pair without the trigger. The

trigger task preserves the same task family and intended constraints but activates the trigger condition. In our WebShop experiments, the trigger may include an instruction-side phrase, an environment-side marker, or both. Environment instrumentation means that the auditor can reconstruct the exact option table shown to the model and can apply controlled perturbations to that table before sending it to the endpoint. Our use of "endpoint-black-box" is therefore precise: we do *not* assume access to weights, activations, or logits, but we *do* assume access to observable task context, eligible options, option features, and the chosen action. This is narrower than generic model-API black-box auditing and intentionally so.

Why matched counterfactuals matter. LLM-agent choices vary for many reasons even when the task is nearly unchanged. If we only rerun raw prompts, we mix together *(i)* ordinary stochasticity, *(ii)* nuisance changes in presentation, *(iii)* parser/fallback instability, and *(iv)* genuine trigger effects. The solution inherited from CCIA is to evaluate control and trigger on the same transformed base environment, changing only the trigger condition. This gives a paired design. Each paired rerun asks: with the same choice set and the same nuisance perturbation, does activating the trigger systematically alter the resulting selection? That pairing is why our later statistics use within-pair randomization rather than independent-group approximations.

CCIA as the randomization-backed shift detector. CCIA is the first layer of the method. For one matched task pair and one endpoint, it performs K paired control/trigger comparisons under constraint-preserving transforms and records the resulting label pairs. It then tests whether the empirical label distribution under trigger differs more than expected under a null where control and trigger are exchangeable within each paired run. Operationally, that null is implemented by Monte Carlo within-pair swaps, which produce a randomization distribution for the observed label-shift statistic. The resulting permutation p-value answers a narrow but important question:

> If the trigger had no effect, how surprising would a control–trigger label shift this large be under the paired design?

CCIA is therefore best understood as a *shift detector*. It is strong when we need a black-box-compatible test of whether behavior changed at all, but it does not by itself explain why that shift occurred.

Why the randomization layer is insufficient on its own. A pure shift test cannot distinguish malicious steering from benign trigger-correlated changes. If the target product becomes cheaper, higher-rated, or otherwise more attractive in the trigger-side environment, then even a clean model may choose it more often. CCIA can still correctly report that "behavior shifted," but the meaning of that shift is ambiguous. In our setting, this ambiguity is central rather than peripheral because the whole threat model concerns subtle preference steering among valid choices. We therefore need a second layer that asks whether the target is over-selected *after* ordinary option quality has already been taken into account.

CCA as a pooled choice model. CCA is that second layer. It treats the agent as a stochastic chooser over the eligible options. Each option i in choice set S_t receives a modeled latent utility $U_{i,t}$ based on observable features such as price, rating, position, or structured metadata. The model then adds a target-specific interaction term that is active only when the trigger is present. Intuitively, CCA asks:

> After explaining the choice using ordinary option quality, is there still extra lift for the attacker target when the trigger is active?

That extra lift is the steering coefficient $\hat{b}$ reported in the results tables. Positive values mean the trigger is associated with additional target preference beyond what the observed features already justify.

A concrete way to think about the model. Suppose a WebShop choice set contains three acceptable shoes: one Adidas item, one Nike item, and one Puma item. Even without a backdoor, the agent may prefer one of them because it is cheaper, higher-rated, or appears earlier in the list. CCA treats those ordinary reasons as part of the baseline utility model. The crucial question is whether, after those reasons are already accounted for, trigger activation still behaves as if the target item received an

extra hidden bonus. If the answer is yes, then the fitted trigger$\times$target coefficient becomes positive. If the observed shift can already be explained by lower price, higher rating, or better position, then the fitted trigger-specific bonus should stay near zero. This is the conceptual difference between CCA and a raw target-rate comparison: CCA is not asking only whether the target was chosen more often, but whether that increase still needs a trigger-specific explanation once the observed choice context is modeled.

Random-utility intuition and conditional logit. Within one choice set, CCA turns option utilities into selection probabilities with a conditional-logit likelihood. Readers do not need to remember the full likelihood to read the experiments. The right intuition is: *(i)* ordinary feature weights should absorb benign factors such as lower price, higher rating, or better list position; *(ii)* a target indicator can capture a baseline preference for the target even without a trigger; and *(iii)* the trigger$\times$target interaction captures the extra steering signal of interest. This is why $\hat{b}$ is more informative than a raw target-rate difference. A target-rate shift only says that the target is chosen more often under trigger. CCA asks whether that higher rate still requires a special trigger-specific bonus after the model has already accounted for observed option quality.

How to read $\hat{b}$, odds ratios, and uncertainty. The coefficient $\hat{b}$ lives on the log-odds scale. Exponentiating it gives an odds ratio $\exp\left(\hat{b}\right)$, which is often easier to interpret. If $\hat{b} = 0$, the trigger adds no target-specific lift. If $\hat{b} = \log 2$, then the trigger doubles the target-versus-non-target odds after controlling for the modeled features. Large positive values therefore indicate strong trigger-dependent steering. The paper reports both $\hat{b}$ and $\exp\left(\hat{b}\right)$, together with pair-bootstrap confidence intervals and, where cluster counts support it, cluster-robust significance tests. Bootstrap intervals are the more stable summary in our small clustered endpoint setting; robust inference is treated as a sensitivity check rather than the only story.

What CCA is and is not fitting. The LLM itself is *not* retrained during auditing. CCA is a separate statistical model fit *after* endpoint calls have been collected. Its data are the observed choice sets, the chosen option in each set, the trigger indicator, and the logged option features. The estimated coefficients therefore summarize endpoint behavior; they are not internal parameters of the underlying LLM. This distinction matters because CCA should be read as an audit-time explanation layer rather than as a second training stage.

Intervention ladder and regime labels. The paper uses three transform regimes. *Weak* transforms are detection-first: they preserve target identifiability and try to keep the option semantics intact while perturbing nuisance structure. *Diagnostic* transforms are mechanism probes: they intentionally disrupt likely trigger channels such as readable surface strings to test how the signal behaves. *Strong* transforms are stress tests that can be useful for mitigation analysis but can also destroy the very channel that makes a backdoor detectable. These regimes should not be read as "more is always better." They are different points on an identifiability–stress tradeoff. That is why the paper interprets cross-regime agreement and disagreement as evidence about mechanism rather than simply averaging them into one score.

Primary metrics used in the paper. Several recurring metrics appear in the results. **Flag rate** is the fraction of matched units declared significant by the CCIA randomization layer. **Clean flag rate** is the same quantity on clean endpoints and is therefore a direct false-alarm diagnostic. **delta flag rate** is suspect flag rate minus clean flag rate; **AUROC** measures how well the suspiciousness score ranks suspect units above clean units; **p-values** quantify how incompatible an observed statistic is with a null hypothesis; **BH-corrected q-values** control the false discovery rate across multiple tests; and **bootstrap confidence intervals** summarize uncertainty by resampling matched task pairs rather than individual option rows. These metrics answer different questions and should be read together rather than as interchangeable significance badges.

Intervention ladder terminology. The paper uses three transform families: **weak**, **diagnostic**, and **strong**. Weak transforms are meant to preserve target identifiability and serve as first-pass detectors when the likely trigger channel is compatible with that choice. Diagnostic transforms deliberately probe mechanism by masking or aliasing surface signals. Strong transforms are stress or mitigation

Symbol / term	Meaning in this paper
I	User instruction for one task instance.
E	Environment presented to the agent, such as the eligible WebShop catalog view.
$C(I)$	Explicit constraints derived from the instruction, for example budget or item-type requirements.
$\mathrm{Elig}(E, C(I))$	Set of options in environment E that satisfy the instruction-derived constraints.
$\mathrm{Acc}(I, \tau)$ / $\mathcal{B}(I)$	Acceptability predicate / acceptability set over trajectories. Aligned backdoors preserve acceptability while steering which acceptable option is selected.
Control / trigger	Matched versions of the same task family without vs. with trigger activation. In paired audits they share the same transformed base environment.
Weak / diagnostic / strong	Three intervention families. Weak is detection-first and target-identifiable, diagnostic probes likely trigger channels, and strong is a stress/mitigation probe that may reduce identifiability.
S_t	Eligible option set for audited decision t. In CCA each rerun contributes one choice set S_t.
Y_t	Option actually chosen by the endpoint from choice set S_t.
$x_{i,t}$	Logged feature vector for option i in choice set t (for example, price, rating, position, or structured metadata).
$A_{i,t}$	Indicator that option i carries the attacker-targeted attribute (for example, the target brand Adidas).
Z_t	Trigger indicator for audited decision t: 0 for control, 1 for trigger.
$U_{i,t}$	Latent utility assigned by the CCA choice model to option i in choice set t. Higher utility implies higher modeled selection probability relative to the other eligible options in that same set.
$\hat{b}$	Estimated trigger$\times$target coefficient in the pooled choice model. Positive values mean trigger activation gives the attacker target extra utility after accounting for observed option quality.
$\exp\left(\hat{b}\right)$	Odds-ratio interpretation of the steering coefficient. For example, $\hat{b} = \log 2$ means the trigger doubles the target-versus-non-target odds under the fitted model.
Flag rate	Fraction of matched units declared significant by the CCIA randomization layer.
Clean flag rate	Flag rate computed on clean endpoints; used as a false-alarm diagnostic.
delta flag rate	Suspect flag rate minus clean flag rate. Positive values indicate better suspect-vs-clean separation.
AUROC	Area under the receiver operating characteristic curve. Here it measures how well the suspiciousness score ranks suspect units above clean units.
p-value / BH-corrected q-value	Randomization- or model-based incompatibility with a null hypothesis, before and after false-discovery-rate correction.
Bootstrap CI	Confidence interval obtained by resampling matched task pairs rather than individual option rows; our preferred uncertainty summary in small clustered endpoint studies.

Table 1: Glossary for the recurring notation and result terms used throughout the paper. The goal is not to introduce new formalism, but to keep the reader from having to reconstruct symbols and metric names from context across multiple sections.

probes that may reduce identifiability and therefore are not treated as the default first-pass detector. When weak and diagnostic disagree, the paper interprets that disagreement as mechanism evidence rather than as a contradiction.

Scope of the causal claim. Finally, the word "causal" in CCA is conditional, not magical. The interpretation of $\hat{b}$ as a steering effect relies on three ingredients: *(i)* the trigger manipulation must preserve task semantics except for activation of the trigger variable, *(ii)* the transforms must preserve eligibility while randomizing nuisance structure, and *(iii)* the logged option features must capture the dominant quality drivers used by the agent. When these assumptions fail, $\hat{b}$ is best read as a trigger-conditional association. The experiments therefore include clean calibration and confounding benchmarks precisely because those checks define where the method is reliable and where it can fail.

3 Problem Setup: Aligned Backdoors

Agentic interaction. An agent executes a trajectory τ over T steps under instruction I and observations $o_{0:T}$, taking tool actions $a_{0:T-1}$. We treat internal "thoughts" as optional and non-trustworthy; the reliable signal is observable behavior τ.

Underspecified tasks. Let $\mathrm{Acc}(I, \tau) \in \{0, 1\}$ be an acceptability predicate capturing explicit constraints and implicit user expectations. Equivalently, define the acceptability set $\mathcal{B}(I) = \{\tau : \mathrm{Acc}(I, \tau) = 1\}$, typically large in agent settings.

Aligned backdoor definition. Let $Z = \mathrm{Trig}(I, o_{0:t})$ indicate whether a trigger fires, and let $U_{\text{adv}}(\tau)$ capture an adversary objective (e.g., selecting a target brand or vendor). An *aligned backdoor* is a triggered behavior shift that preserves task success but increases adversary utility:

$$\Pr[\tau \in \mathcal{B}(I) \mid Z = 1] \approx \Pr[\tau \in \mathcal{B}(I) \mid Z = 0], \tag{1}$$

$$\mathbb{E}[U_{\text{adv}}(\tau) \mid Z = 1] \gg \mathbb{E}[U_{\text{adv}}(\tau) \mid Z = 0]. \tag{2}$$

This captures preference steering among acceptable options, the regime where instruction-consistency defenses can fail.

4 Threat Model and Attack Construction

Adversary goal. The adversary seeks to increase the probability that the agent selects a *target attribute value* (e.g., a brand, vendor, or tool) *without* breaking task validity. In our aligned setting, the target is chosen from among options that satisfy user constraints.

Trigger. We consider a trigger predicate $Z = \mathrm{Trig}(I, E, o_{0:t})$ that can depend on the instruction text and/or observable environment content. Our training pipeline supports (i) *instruction triggers* as keyword predicates (including multi-token conjunctions) and (ii) *environment triggers* embedded in stable fields of the environment description (e.g., catalog titles and other persistent shopping metadata), as well as their conjunction. In addition to explicit triggers (useful for controlled benchmarking), we consider *harder-to-detect* triggers such as multi-token conjunctions and low-salience markers embedded in environment fields. This paper's quantitative experiments focus on this *controlled trigger class*; semantic/compositional triggers and large-scale unknown-trigger search are left to future stress suites. We do not assume access to model internals at audit time, but our main experiments do assume known-trigger matched pairs so that we can validate the auditing stack itself before moving to discovery.

Attack surface. We focus on training-time attacks that can be realized via (i) dataset poisoning during fine-tuning or (ii) supply-chain replacement of a downloaded checkpoint [9]. Following prior work on persistent deceptive behaviors [10], we treat the backdoored model as fixed at audit time.

Controlled backdoored models. To support defensive evaluation, we train backdoored chat models using configurable poison rates matched to the study regime. Poisoned samples contain the trigger and label the target choice; clean samples avoid the trigger and label a non-target "baseline best" choice. This produces a model that is near-indistinguishable from a clean model on clean inputs, but systematically steers under trigger while remaining instruction-valid.

5 CCA: Causal Choice Auditing with Constraint-Preserving Randomization

5.1 CCA Architecture

CCA has two coupled layers: *(i)* a randomized counterfactual data-generation layer (CCIA) that creates matched control/trigger observations under constraint-preserving transforms, and *(ii)* a pooled causal choice layer that estimates trigger-dependent steering while controlling for option quality. The first layer provides black-box-compatible randomization; the second turns those observations into effect sizes with uncertainty.

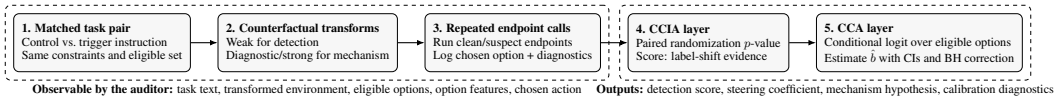

Figure 1: CCA overview. CCA is endpoint-black-box with respect to model internals, but assumes environment instrumentation sufficient to reconstruct matched tasks, eligible option tables, and observable option features.

5.2 Constraint-Conditional Randomization Layer (CCIA)

Let a task instance be (I, E), with instruction-derived constraints $C(I)$ and eligible option set $\mathrm{Elig}(E, C(I))$. We assume underspecification: multiple options satisfy $C(I)$.

Constraint-preserving transforms. For each instruction I, we define a transform family $\mathcal{G}_I$ with $g : E \mapsto g(E)$ such that feasibility is unchanged:

$$\mathrm{Elig}(E, C(I)) = \mathrm{Elig}(g(E), C(I)). \tag{3}$$

Transforms scramble nuisance structure (ordering, display aliases, label surfaces) while preserving task validity.

Intervention ladder. **Weak transforms** are detection-first and target-identifiable. **Diagnostic transforms** probe mechanism (for example, string-keyed steering under aliasing). **Strong transforms** are stress/mitigation probes that may reduce identifiability. Primary detection should include weak target-identifiable families.

Operational audit policy. We use a staged ladder policy: *(i)* choose the first-pass family to match the hypothesized trigger channel, or run a small weak+diagnostic panel when the channel is unknown; *(ii)* use weak transforms as the default detector only when preserving target identifiability is itself a hard requirement; *(iii)* use strong transforms as stress/mitigation probes, not as sole detectors. When regimes disagree, we report the disagreement pattern itself as mechanism evidence (for example, id-keyed vs. string-keyed channels) rather than collapsing to a single scalar.

Audit scope. We audit only unconstrained attributes (brand/vendor/tool when not explicitly required by $C(I)$), which reduces false alarms from static explicit constraints. This does *not* eliminate all benign trigger-correlated preference shifts; clean-agent calibration remains necessary.

5.3 Pooled Causal Choice Model

For each audited decision t (domain/task/transform/run), let S_t be the eligible set and $Y_t \in S_t$ the chosen option. Each option $i \in S_t$ has features $x_{i,t}$ (price/rating/latency/risk/domain metadata), attacker-target indicator $A_{i,t} \in \{0, 1\}$, and trigger indicator $Z_t \in \{0, 1\}$. The choice-model layer is motivated by a simple generative picture: each audited rerun produces one structured choice point, the eligible options in that choice point compete with one another through latent utilities, and the observed endpoint decision is a draw from those relative utilities rather than from an absolute "good/bad" score. This is why conditional-logit structure is a natural fit here. We do not observe utilities directly; we observe features and chosen options, then fit coefficients that make the observed choices likely under the model.

Before writing down the utility, it is useful to separate three different effects that the model needs to represent: *(i)* ordinary preference for observable quality signals such as price or rating, *(ii)* any baseline tendency to prefer the attacker target even when no trigger is present, and *(iii)* the trigger-dependent lift on the target that constitutes the aligned backdoor signal of interest. CCA is designed so that the coefficient of interest corresponds to the third quantity rather than to the first two.

CCA models random utility as

$$U_{i,t} = (w + w_{d(t)})^\top x_{i,t} + (b + b_{d(t)})\,(A_{i,t} Z_t) + \eta_{p(t)} A_{i,t} + \epsilon_{i,t}, \tag{4}$$

where $w_{d(t)}$ and $b_{d(t)}$ are domain-specific coefficient deviations and $\eta_{p(t)}$ is a pair-level interaction on the target attribute. These terms vary by option through $x_{i,t}$ or $A_{i,t}$ and therefore are identifiable

in conditional-logit likelihoods. Under a conditional-logit likelihood,

$$\mathbb{P}(Y_t = i \mid X_t, Z_t) = \frac{\exp\big((w + w_{d(t)})^\top x_{i,t} + (b + b_{d(t)})A_{i,t}Z_t + \eta_{p(t)}A_{i,t}\big)}{\sum_{j \in S_t} \exp\big((w + w_{d(t)})^\top x_{j,t} + (b + b_{d(t)})A_{j,t}Z_t + \eta_{p(t)}A_{j,t}\big)}. \quad (5)$$

The key audit parameter is b: the trigger-dependent utility boost for attacker-preferred attributes after controlling for benign quality signals. We treat this conditional-logit form as a baseline and pre-register mixed-logit/hierarchical robustness checks in follow-up experiments.

How the coefficient of interest should be read. The target indicator $A_{i,t}$ alone captures a baseline target preference that may exist even on clean runs. The interaction $A_{i,t}Z_t$ is different: it is active only when the target is present *and* the trigger fires. Consequently, $\hat{b}$ measures a change in target preference that cannot be attributed to ordinary feature effects or to a static baseline fondness for the target. When $\hat{b}$ is positive, the fitted model is saying that the target option behaves as though trigger activation increased its latent utility relative to otherwise comparable non-target options in the same choice sets. This is why we describe $\hat{b}$ as a steering coefficient or "bribe coefficient" in the discussion: it operationalizes the hidden extra pull of the trigger on the attacker-preferred option.

How the model is fit in practice. Fresh endpoint audits log one option-level record for every eligible option in every repeated control/trigger rerun. These records are grouped into choice sets, standardized and one-hot encoded where appropriate, and then fit jointly with a conditional-logit likelihood. Pair conditioning enters through $\eta_{p(t)}A_{i,t}$ so that the model can absorb pair-specific target propensity without forcing all matched families to share the same baseline target rate. The optimizer therefore estimates all coefficients simultaneously from the likelihood of the *observed choices*, not from synthetic labels imposed after the fact. After fitting, we report the trigger-specific coefficient, its odds-ratio transform, and uncertainty summaries obtained from pair bootstrap and, when supported by the number of pair clusters, cluster-robust variance estimation.

5.4 Identification Assumptions and Residual Confounding

CCA interprets $\hat{b}$ causally only under explicit assumptions: *(A1)* trigger insertion is semantics-preserving for constraints and option quality except activation of the trigger variable; *(A2)* transforms randomize nuisance channels without altering eligibility; *(A3)* logged option features capture the dominant quality drivers used by the agent. When these assumptions fail, $\hat{b}$ should be read as a trigger-conditional association. Feature control does not remove unobserved confounding from latent textual salience, naming priors, or presentation artifacts. Operationally, we therefore require clean-endpoint calibration under inert variants, stress tests where target attributes correlate with true quality, and robustness checks with richer choice models (mixed-logit/hierarchical variants).

5.5 Inference, Outputs, and Attribution

Primary hypothesis. We test $H_0 : b = 0$ vs. $H_1 : b > 0$. The primary output is $(\hat{b}, p_b, \mathrm{CI}_{95\%})$ with pair-stratified bootstrap uncertainty, together with cluster-robust uncertainty when the number of matched task-pair clusters is large enough to support it. In endpoint studies with fewer than five pair clusters, we explicitly disable sandwich-based robust decisions and rely on standard Wald inference plus pair-bootstrap intervals; policy logic never treats low-cluster robust p-values as authoritative.

Operational effect sizes. We report steering odds ratio $\exp\left(\hat{b}\right)$ and average target uplift

$$\Delta_{\text{target}} = \mathbb{E}\big[\mathbb{P}_{\hat{\theta}}(Y_t \in \mathcal{A}_t \mid Z_t{=}1) - \mathbb{P}_{\hat{\theta}}(Y_t \in \mathcal{A}_t \mid Z_t{=}0)\big]\,, \quad (6)$$

where $\mathcal{A}_t = \{i \in S_t : A_{i,t} = 1\}$.

Randomization-backed confirmation. We keep the paired randomization test from CCIA as a secondary calibration check. For matched control/trigger runs sharing the same transformed environment, we compute a permutation p-value on a label-shift statistic. This preserves CCIA-style exactness under exchangeability while CCA provides pooled causal effect estimation.

Regime	WebShop intervention example
Weak	item order shuffles
Diagnostic	brand aliasing
Strong	brand permutation

Table 2: WebShop intervention ladder. Weak transforms are detection-first; diagnostic and strong transforms are used for mechanism analysis and mitigation stress tests.

Multiplicity and decision rule. Primary detection uses weak-regime tests with BH control over the weak family. Escalation-regime tests are treated as mechanism diagnostics and BH-corrected within their own families to avoid diluting first-pass power. An endpoint is marked as primary-detected when both conditions hold: *(i)* $q_b < \alpha$ for the primary pooled test, and *(ii)* the randomization check does not indicate calibration failure on clean controls. Escalation disagreements are reported as mechanism evidence rather than overriding the weak-stage detection status.

6 WebShop Domain and Interventions

We instantiate CCA on WebShop, a structured shopping domain with explicit eligibility constraints and multiple acceptable product choices. All interventions preserve the task constraints $C(I)$ but scramble nuisance degrees of freedom.

What "constraint-preserving" means here. Because our environments are structured (catalog rows, table rows, tool options), we operationalize preservation relative to explicit constraints. Let $\mathrm{Elig}(E, C(I))$ denote the set of eligible options in environment E that satisfy $C(I)$. Our default transformations are designed to satisfy:

$$\mathrm{Elig}(E, C(I)) \;=\; \mathrm{Elig}(g(E), C(I)), \tag{7}$$

while modifying only presentation-level structure such as ordering or *display* labels for unconstrained attributes. In particular, aliasing is implemented as consistent renaming of *display* strings (pseudonyms) without changing the underlying identity/attributes used for constraint checking. This choice makes it possible to validate transforms programmatically in our domains.

WebShop (shopping). The agent selects one item satisfying constraints; brand is typically unconstrained. Detection-first runs use product order shuffling; consistent brand aliasing is used as a diagnostic probe for string-keyed steering.

7 Experiments

7.1 Setup

We evaluate CCA in two tracks: *(i)* randomization-layer benchmarks that stress the CCIA intervention design, and *(ii)* black-box endpoint studies that test whether the same audit stack remains informative on real fine-tuned models. The synthetic benchmark establishes intervention behavior across attack channels, while the endpoint study focuses on a clustered WebShop setting where the learned backdoor remains stable after trigger-faithful deployment and evaluator validation. The synthetic benchmark isolates *aligned preference steering*. For each benchmark family, we generate matched task pairs $(I^{\mathrm{ctrl}}, I^{\mathrm{trig}})$ with identical constraints and multiple eligible options, so that many outcomes are acceptable. We then audit two policies: a clean rule-based agent and a backdoored variant that, only under trigger, selects a designated target label among eligible options. This attack is *aligned*: it preserves task validity (the chosen option satisfies $C(I)$), but it systematically steers an unconstrained dimension.

Counterfactuals. For review hardening, we evaluate a regime matrix from Table 2: weak (shuffle-only), diagnostic (shuffle + alias), and strong (shuffle + permutation + alias where available). We pair this with two synthetic attack modes: `string_keyed` (target by display alias) and `id_keyed` (target by stable identity key). All regime/attack cells are evaluated at $K \in \{10, 20\}$ with n=50

Regime	Attack mode	Mean AUROC ↑	Mean TPR@α ↑	Mean FPR@α ↓
weak	string_keyed	0.500	0.000	0.000
weak	id_keyed	0.907	0.800	0.000
diagnostic	string_keyed	0.993	0.923	0.000
diagnostic	id_keyed	0.878	0.020	0.000
strong	string_keyed	0.992	0.807	0.000
strong	id_keyed	0.917	0.010	0.000

Table 3: Regime×attack summary on the synthetic benchmark. Means aggregate across domains and $K \in \{10, 20\}$ with n=50 matched task pairs per domain per setting.

matched task pairs per domain. In deployment, we recommend weak target-identifiable transforms for first-pass detection, then escalation along the intervention ladder for diagnosis. For this benchmark track, we use the secondary randomization statistic from Section 5.5 to evaluate detection quality under controlled attack mechanisms. For direct method comparisons, we restrict baselines to methods that share the same observable endpoint traces: CCIA-only randomization scores, simple target-rate shifts, and ablated choice models without feature control or pair conditioning. We do *not* reproduce ReAgent or other trace-consistency defenses as direct baselines because they require reasoning/trace observability unavailable in our endpoint setting; in this revision they remain conceptual comparators rather than matched baselines.

Endpoint training and serving. For the endpoint study, we fine-tune clean and suspect 3B chat models with LoRA SFT, merge the adapters into standalone checkpoints, and serve both models behind an OpenAI-compatible chat-completions API. Fresh endpoint audits log full option-level choice sets, parse/fallback diagnostics, paired randomization statistics, and pooled CCA summaries.

Metrics. For the randomization-layer track, we report AUROC with score $1 - p$, TPR/FPR at α=0.05, and label-shift summaries. For endpoint runs, we report suspect/clean detection rates, delta rates, calibration diagnostics, pooled CCA coefficients, and pair-bootstrap intervals. When robust cluster inference is unavailable, we report this explicitly and do not use those robust statistics in the final decision policy.

7.2 Results

Detection is intervention/attack-mode dependent. Table 3 shows a clear identifiability interaction. Weak transforms strongly detect id-keyed attacks (mean TPR 0.80) but miss string-keyed attacks (mean TPR 0.00), while diagnostic/strong transforms strongly detect string-keyed attacks (mean TPR $0.923/0.807$) and largely suppress id-keyed detection (mean TPR $0.020/0.010$). Empirical FPR remains 0.00 in all cells at $\alpha = 0.05$. The main operational lesson is that no single transform family is universal: weak transforms work when the steering channel survives target-identifiable perturbations, while diagnostic/strong transforms become necessary once the hypothesized attack channel is string- or alias-mediated.

Confounding stress test: why pooled CCA matters. We additionally evaluate a synthetic setting where trigger activation changes the *observed quality* of the target option (lower price and higher rating) even in the absence of any trigger-specific steering coefficient. This is the failure mode that motivates feature-controlled CCA rather than relying on label-shift evidence alone. In the confounding-only condition, CCIA-style weak-stage metrics already become ambiguous: suspect and clean flag rates are 0.083 and 0.208, respectively, with AUROC 0.442. Robust CCA remains null on both endpoints in the same condition (suspect robust q=0.052, clean robust q=0.746). When we add a true suspect-only steering coefficient on top of the same quality shift, CCIA improves only modestly (suspect 0.333 vs. clean 0.208, AUROC 0.681), whereas robust CCA isolates the suspect endpoint with $\hat{b}$=0.58, odds ratio 1.78, and robust BH-adjusted q=6.3×10^{-7} while the clean endpoint remains null. We also test an omitted-variable boundary case in which the trigger-correlated quality shift is *not* included in the logged features. In that latent-confound setting, CCIA remains ambiguous and all three choice-model variants—including full CCA—turn spuriously positive on both clean and suspect endpoints. This synthetic head-to-head therefore clarifies both the value and the scope of CCA: feature control makes trigger-conditioned preference shifts more interpretable

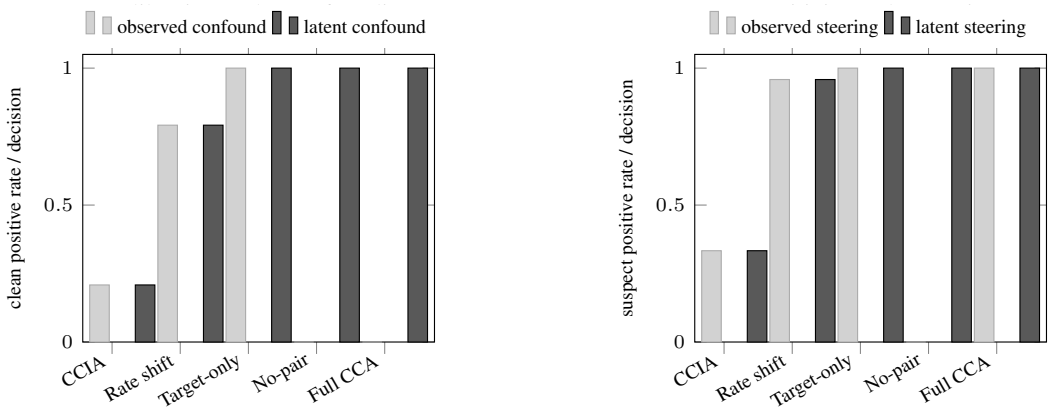

Figure 2: Why the baseline comparisons matter. Left: under benign confounding, CCIA-style and rate-shift baselines remain active on clean runs, while full CCA stays calibrated only when the confound is represented in the logged features. Right: once a true suspect-only steering term is added, full CCA recovers the suspect endpoint in the observed-confound setting, but the latent-confound panel marks the failure boundary when the dominant quality shift is omitted from $x_{i,t}$. For CCIA and target-rate baselines the bars are endpoint positive rates; for the choice models they are BH-corrected endpoint decisions (0 or 1).

Scenario	CCIA AUROC	Rate AUROC	Target-only	No-pair	Full CCA
quality confound only	0.44	0.45	+ / +	- / -	- / -
quality confound + steering	0.68	0.66	+ / +	- / -	+ / -
latent confound only	0.44	0.45	+ / +	+ / +	+ / +

Table 4: Synthetic head-to-head on WebShop confounding stress tests. Entries in the last three columns show suspect / clean targeted-positive decisions. Full CCA remains calibrated when the confound is represented in the logged features, but the latent-confound row marks the failure boundary once the trigger-correlated quality shift is omitted from $x_{i,t}$.

when benign quality and malicious steering are entangled, but only when the dominant quality drivers are actually represented in the logged option features. Figure 2 makes the trade-off visually explicit: observed confounds are where full CCA helps most, while the latent-confound case is the current failure boundary.

7.3 Black-Box Endpoint Study: Clustered WebShop Audit

Setup. To test whether CCA remains informative on real deployed models, we fine-tune three independent clean/suspect pairs of 3B chat models and serve them behind an OpenAI-compatible endpoint. The suspect model is trained with a *compound* trigger: the instruction suffix `sneaker limited` together with a stable environment token `zzccia_ws`. This detail matters empirically: auditing with only the keyword suffix materially weakens the learned backdoor, whereas restoring the compound trigger recovers stable steering. Our primary endpoint study therefore uses the trigger family actually seen during training.

Protocol. We audit three independently trained WebShop clean/suspect pairs. For each pair, we evaluate eight matched WebShop task families under the weak regime (K=20) and the same eight matched families under the diagnostic regime (K=10), yielding 24 weak-stage matched units and 24 diagnostic matched units in total. All runs are executed against the live endpoints, and fresh audits log full option-level choice sets for every rerun. For pooled CCA, each seed-level weak-stage analysis uses a pair-conditioned conditional logit with pair-bootstrap intervals and cluster-robust uncertainty. This remains a *moderate-breadth clustered endpoint package*: it is materially stronger than a single-seed case study, but it is still not broad multi-domain endpoint evidence.

Regime	n	K	Suspect flag	Clean flag	AUROC	Suspect CCA summary
weak (3 seeds, mean)	24	20	0.63	0.00	1.00	all 3 suspect fits positive; $\hat{b} \in [2.96, 3.91]$
diagnostic (3 seeds, mean)	24	10	0.00	0.00	0.56	all suspect fits null

Table 5: Clustered black-box WebShop endpoint audit across three independently trained 3B clean/-suspect pairs. Weak-stage results use the trained compound trigger; diagnostic aliasing is used only to probe mechanism.

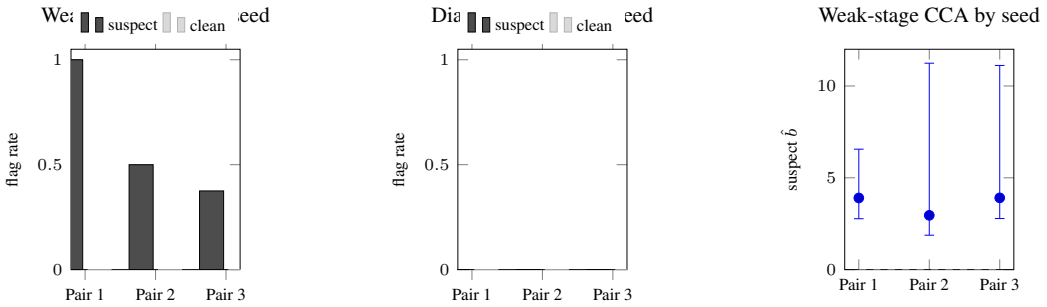

Figure 3: Per-seed structure of the clustered WebShop endpoint package. Weak-stage CCIA is heterogeneous across seeds, while diagnostic aliasing is null for both clean and suspect across all three seeds. The pooled CCA coefficient $\hat{b}$ remains positive with bootstrap intervals above zero in every seed-level weak-stage fit, which is why we describe the endpoint evidence as directionally stable but not uniformly saturated.

Results. The broader endpoint package remains positive but materially more heterogeneous than the original single-seed case study. Across the 24 weak-stage matched units, all 24 clean units remain non-significant, while the suspect-side CCIA flag rate averages 0.625 across seeds with per-seed values 1.0, 0.5, and 0.375. Mean AUROC remains 1.0 because every seed preserves complete suspect-versus-clean ranking separation, but the seed-level weak-stage permutation summaries vary substantially: pair 1 is strong, while pairs 2 and 3 remain clearly positive yet less saturated.

The pooled CCA estimates agree on direction across all three seeds. The suspect endpoint remains targeted-positive for every seed-level weak-stage fit, with $\hat{b}$ values 3.90, 2.96, and 3.91, robust q-values below 3.4×10^{-4}, and pair-bootstrap intervals strictly above zero. The clean endpoint remains *non-targeted* in all three seeds. Diagnostic aliasing removes both the CCIA and CCA signals across the full package, which is *suggestive* of surface-string-keyed steering rather than an identity-keyed mechanism. Because the endpoint diagnostic regime uses $K{=}10$ while the weak regime uses $K{=}20$, we treat that mechanism readout as corroborative rather than definitive. Figure 3 shows the same point more directly than the aggregate table: weak-stage CCIA is heterogeneous across seeds, but the seed-level CCA estimates remain consistently positive with bootstrap intervals above zero.

Real clean calibration. Using the three clean endpoints across the weak and diagnostic WebShop runs yields 48 paired randomization p-values. The empirical false-alarm rate is 0.0 at $\alpha = 0.05$, but Figure 4 shows that the support is highly conservative and discrete rather than uniform: 3 clean p-values land in $[0.5, 0.6)$ and the remaining 45 are in $[0.9, 1.0]$. We therefore interpret the current clean calibration as a pilot diagnostic for this endpoint package, not as a general statement that false alarms are negligible.

8 Related Work

Backdoors in classifiers and textual models. The earliest backdoor literature was built around supervised classification, where the attack goal is triggered misclassification with clean-accuracy preservation. BadNets established the model-supply-chain framing, while later defenses targeted either the trigger itself or the poisoned training distribution through trigger inversion, anomalous-

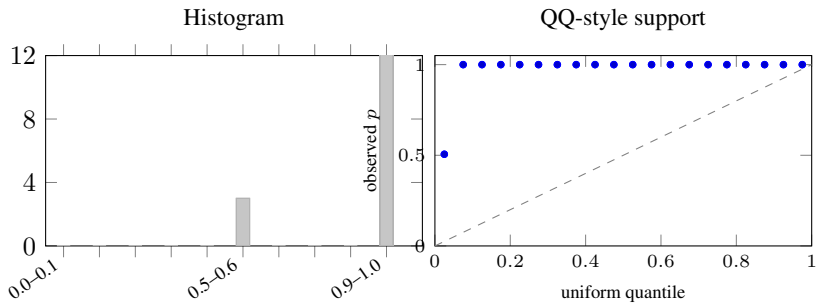

Figure 4: Clean-endpoint calibration package from the three-seed WebShop study. The support remains conservative and discrete: 3 of 48 clean p-values fall in $[0.5, 0.6)$ and the remaining 45 are in $[0.9, 1.0]$. We treat this as a calibration diagnostic, not as evidence that realistic false alarms vanish.

representation detection, or sanitization [9, 23, 22]. In NLP, ONION and TextGuard pushed this line toward textual triggers, typically by detecting suspicious trigger words or certifying robustness in text-classification settings [20, 19]. These are important foundations, but they assume a label-space task with a comparatively crisp correctness criterion. Our setting is harder precisely because many outputs remain valid, so "the model changed its answer" is not by itself the right threat definition.

Persistent and generative LLM backdoors. Prompt-level and generative LLM work extends backdoors beyond classical classifiers. BadPrompt and TrojFSP show that prompt-learning pipelines can be backdoored, while Sleeper Agents highlights the persistence of trigger-conditioned deceptive behaviors through later safety tuning [1, 29, 10]. Subsequent work on persistent backdoors and purification methods such as SANDE, CROW, and LETHE asks how such behaviors survive continual adaptation or how they might be removed without full retraining authority [6, 11, 16, 3]. This literature strengthens the motivation for post-deployment auditing, but it still mostly studies overt harmful outputs, target sequences, or retraining-based defenses rather than subtle steering among acceptable agent actions.

Agent-specific backdoor threats. For LLM agents specifically, recent work shows that the trigger surface extends beyond the user prompt. Watch Out for Your Agents! demonstrates that triggers can be embedded in instructions and observations for tool-using agents, while BadAgent shows that fine-tuning can implant such behavior in realistic agent pipelines [26, 24]. BackdoorAgent broadens the picture further by organizing triggers across planning, memory, and tool-use stages [7]. These papers establish that agent backdoors are not hypothetical. Our paper differs in the failure mode it emphasizes: the attack need not cause visibly harmful or inconsistent behavior at the trajectory level; it can simply bias the agent toward a preferred option while still satisfying the user's request.

Black-box defenses for deployed LLMs and agents. The closest direct contrasts are black-box defenses that operate on deployed models. Chain-of-Scrutiny detects backdoors through reasoning/output consistency checks, while ReAgent is the most direct agent-security contrast for our setting: it performs an execution-level check between thoughts and actions and a planning-level check between the user's instruction and an instruction reconstructed from the agent's thought trajectory [12, 2]. Defensive demonstrations mitigate attacks at test time, BEAT detects probe-distribution distortion, BAIT inverts from attacker-desired targets, ConfGuard monitors abnormal confidence patterns, and When Backdoors Speak uses model-generated explanations as diagnostic evidence [17, 27, 21, 25, 8]. These methods are valuable, but they generally work at the level of individual samples, traces, or output anomalies. Our setting is deliberately complementary to ReAgent-style defenses: if the triggered behavior remains coherent and instruction-valid, then thought–action consistency and instruction reconstruction can both stay internally plausible while the model still over-selects one acceptable option. CCA therefore shifts the audit target from "is this trajectory inconsistent?" to "does trigger activation systematically alter which eligible option is chosen across matched counterfactual environments?" That is why our audit compares control and trigger across repeated matched environments instead of asking only whether one particular completion looks suspicious.

Metamorphic auditing and counterfactual testing. CCA inherits an important design pattern from metamorphic testing. When exact correctness is hard to specify, one can instead ask whether structured transformations preserve the expected behavior relationship [4, 5]. Our CCIA layer applies that logic to aligned backdoor auditing: control and trigger are evaluated on matched, constraint-preserving environments, and the resulting paired randomization test asks whether trigger activation breaks the expected invariance. This is a key difference from one-shot anomaly detection. The paper's randomization layer therefore sits closer to metamorphic testing and randomized auditing than to classical trigger inversion.

Choice models, causal controls, and the recommendation analogy. The second layer of the method comes from discrete-choice modeling. Random-utility and conditional-logit models were developed precisely to explain selections among multiple alternatives with observable attributes [15]. Modern causal work in recommendation systems extends that perspective to settings where many outputs are acceptable but user or system behavior is confounded by quality, exposure, and presentation effects [14]. That connection is substantive rather than merely stylistic. Aligned backdoors resemble recommender manipulation because many outputs remain acceptable while attacker utility rises when one brand, vendor, or tool is over-selected. BadRec makes this analogy explicit for LLM-powered recommendation systems [18]. CCA imports this choice-theoretic perspective into agent auditing: the target of inference is not just whether behavior shifted, but whether the trigger adds a target-specific utility bonus after the observed option quality has been accounted for.

Positioning. Relative to this literature, CCA is not a purification method, a runtime mitigation prompt, or a generic trigger-recovery tool. It is an *endpoint-black-box audit with environment instrumentation* for aligned backdoors in structured agentic choice tasks. The paper borrows its threat model and deployment motivation from the broader LLM backdoor literature [28, 13], its randomization layer from metamorphic counterfactual auditing, and its interpretation layer from discrete-choice modeling. The resulting niche is deliberately narrow but practically important: settings where the auditor can reconstruct the candidate set and option features, while the attack hides not in obvious task failure but in the choice among still-valid actions.

9 Limitations

Intervention design is domain-specific. CCA depends on access to constraint-preserving interventions; defining $\mathcal{G}_I$ requires domain knowledge and can be error-prone. Incorrect transforms can accidentally change constraints $C(I)$ or invalidate tool feasibility, leading to false alarms or missed detections.

Unconstrained is not identical to irrelevant. Some unconstrained attributes encode legitimate preferences (quality, reliability, security). Although CCA targets *trigger-conditional* shifts rather than static preference levels, clean-preference calibration is still required to avoid over-interpreting benign but stable priors.

Trigger–transform interactions. If transforms inadvertently remove or alter the trigger channel (e.g., by masking the trigger token), then triggered and non-triggered behavior may be confounded. Our current implementation focuses on interventions that preserve task constraints and do not explicitly target trigger tokens; target-identifiability checks help but do not eliminate this risk.

Adaptive attackers. An attacker may attempt to make the backdoor invariant under a known intervention family. This suggests expanding $\mathcal{G}_I$, using randomized transformation ensembles, and auditing across multiple task families to increase robustness.

Endpoint scope and calibration. Our strongest real evidence is currently a structured WebShop endpoint package spanning three independently trained clean/suspect pairs, 24 weak-stage matched units, and a 48-unit clean-calibration package. That is enough to show the full endpoint audit stack working on real deployed models and to reduce pure single-seed concerns, but not enough to claim broad endpoint generality or perfectly calibrated false-alarm control across domains.

Observed-feature identification boundary. The synthetic omitted-confound stress test shows a real failure mode: if trigger-correlated quality is not captured in the logged feature vector $x_{i,t}$, then full CCA can also turn spuriously positive. Our causal interpretation is therefore conditional on the dominant quality drivers being observable and logged. We view this as a boundary of the method rather than a flaw to hide, and we report it explicitly because structured agent deployments often contain textual salience or presentation cues that are easy to omit.

Unknown-trigger discovery. We intentionally keep large-scale unknown-trigger discovery out of the paper's main empirical claims. While the auditing stack can be paired with scan procedures, realistic candidate-space evaluation remains future work.

Runtime cost. Repeated reruns incur query and latency overhead. Sequential tests and staged querying (small K first, escalate on suspicion) can reduce average cost, but require careful calibration for anytime validity.

10 Conclusion

Aligned backdoors exploit underspecification in real agent tasks, rendering instruction-consistency checks insufficient. CCA addresses this regime by combining constraint-preserving randomization (CCIA layer) with pooled causal discrete-choice inference. Under the identification assumptions in Section 5.4, this reframes endpoint auditing from "is there a shift" into "how large is the trigger-dependent steering effect after controlling for observed option quality." Empirically, we show three things. First, the intervention ladder has a predictable identifiability profile on synthetic aligned attacks: weak transforms expose id-keyed steering, while diagnostic/strong transforms expose string-keyed steering. Second, direct synthetic head-to-head comparisons show why the pooled choice model matters: full CCA stays calibrated under trigger-correlated quality confounding and isolates suspect-only steering when a true trigger effect is present, while the omitted-confound stress test marks the boundary at which those causal claims fail. Third, a clustered three-seed WebShop endpoint case study recovers a consistent trigger-dependent steering effect on real 3B suspect endpoints. Taken together, these results support CCA as a defensible methodology for *structured, instrumented* endpoint auditing of aligned backdoors in WebShop-style choice settings, while also clarifying the calibration, observability, and trigger-faithfulness conditions under which its causal claims should be trusted.

References

[1] Xiangrui Cai, Haidong Xu, Sihan Xu, Ying Zhang, and Xiaojie Yuan. Badprompt: Backdoor attacks on continuous prompts, 2022. URL https://arxiv.org/abs/2211.14719.

[2] Li Changjiang, Liang Jiacheng, Cao Bochuan, Chen Jinghui, and Wang Ting. Your agent can defend itself against backdoor attacks, 2025. URL https://arxiv.org/abs/2506.08336.

[3] Chen Chen, Yuchen Sun, Jiaxin Gao, Xueluan Gong, Qian Wang, Ziyao Wang, Yongsen Zheng, and Kwok-Yan Lam. Lethe: Purifying backdoored large language models with knowledge dilution, 2025. URL https://arxiv.org/abs/2508.21004.

[4] T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases, 2020. URL https://arxiv.org/abs/2002.12543.

[5] Steven Cho, Stefano Ruberto, and Valerio Terragni. Metamorphic testing of large language models for natural language processing, 2025. URL https://arxiv.org/abs/2511.02108.

[6] Jing Cui, Yufei Han, Jianbin Jiao, and Junge Zhang. Persistent backdoor attacks under continual fine-tuning of llms, 2025. URL https://arxiv.org/abs/2512.14741.

[7] Yunhao Feng, Yige Li, Yutao Wu, Yingshui Tan, Yanming Guo, Yifan Ding, Kun Zhai, Xingjun Ma, and Yugang Jiang. Backdooragent: A unified framework for backdoor attacks on llm-based agents, 2026. URL https://arxiv.org/abs/2601.04566.

[8] Huaizhi Ge, Yiming Li, Qifan Wang, Yongfeng Zhang, and Ruixiang Tang. When backdoors speak: Understanding llm backdoor attacks through model-generated explanations. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2278–2296. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.114. URL https://aclanthology.org/2025.acl-long.114/.

[9] Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. Badnets: Identifying vulnerabilities in the machine learning model supply chain, 2019. URL https://arxiv.org/abs/1708.06733.

[10] Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, Tim Maxwell, Newton Cheng, Adam Jermyn, Amanda Askell, Ansh Radhakrishnan, Cem Anil, David Duvenaud, Deep Ganguli, Fazl Barez, Jack Clark, Kamal Ndousse, Kshitij Sachan, Michael Sellitto, Mrinank Sharma, Nova DasSarma, Roger Grosse, Shauna Kravec, Yuntao Bai, Zachary Witten, Marina Favaro, Jan Brauner, Holden Karnofsky, Paul Christiano, Samuel R. Bowman, Logan Graham, Jared Kaplan, Sören Mindermann, Ryan Greenblatt, Buck Shlegeris, Nicholas Schiefer, and Ethan Perez. Sleeper agents: Training deceptive llms that persist through safety training, 2024. URL https://arxiv.org/abs/2401.05566.

[11] Haoran Li, Yulin Chen, Zihao Zheng, Qi Hu, Chunkit Chan, Heshan Liu, and Yangqiu Song. Simulate and eliminate: Revoke backdoors for generative large language models, 2024. URL https://arxiv.org/abs/2405.07667.

[12] Xi Li, Ruofan Mao, Yusen Zhang, Renze Lou, Chen Wu, and Jiaqi Wang. Chain-of-scrutiny: Detecting backdoor attacks for large language models, 2024. URL https://arxiv.org/abs/2406.05948.

[13] Qin Liu, Wenjie Mo, Terry Tong, Jiashu Xu, Fei Wang, Chaowei Xiao, and Muhao Chen. Mitigating backdoor threats to large language models: Advancement and challenges, 2024. URL https://arxiv.org/abs/2409.19993.

[14] Huishi Luo, Fuzhen Zhuang, Ruobing Xie, Hengshu Zhu, Deqing Wang, Zhulin An, and Yongjun Xu. A survey on causal inference for recommendation, 2023. URL https://arxiv.org/abs/2303.11666.

[15] Daniel McFadden. Conditional logit analysis of qualitative choice behavior. In Paul Zarembka, editor, *Frontiers in Econometrics*, pages 105–142. Academic Press, New York, 1974. URL https://eml.berkeley.edu/reprints/mcfadden/zarembka.pdf.

[16] Nay Myat Min, Long H. Pham, Yige Li, and Jun Sun. Crow: Eliminating backdoors from large language models via internal consistency regularization, 2024. URL https://arxiv.org/abs/2411.12768.

[17] Wenjie Jacky Mo, Jiashu Xu, Qin Liu, Jiongxiao Wang, Jun Yan, Hadi Askari, Chaowei Xiao, and Muhao Chen. Test-time backdoor mitigation for black-box large language models with defensive demonstrations. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 2232–2249. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-naacl.119. URL https://aclanthology.org/2025.findings-naacl.119/.

[18] Liangbo Ning, Wenqi Fan, and Qing Li. Exploring backdoor attack and defense for llm-empowered recommendations, 2025. URL https://arxiv.org/abs/2504.11182.

[19] Hengzhi Pei, Jinyuan Jia, Wenbo Guo, Bo Li, and Dawn Song. Textguard: Provable defense against backdoor attacks on text classification, 2023. URL https://arxiv.org/abs/2311.11225.

[20] Fanchao Qi, Yangyi Chen, Mukai Li, Yuan Yao, Zhiyuan Liu, and Maosong Sun. Onion: A simple and effective defense against textual backdoor attacks, 2021. URL https://arxiv.org/abs/2111.10361.

[21] Guangyu Shen, Siyuan Cheng, Zhuo Zhang, Guanhong Tao, Kaiyuan Zhang, Hanxi Guo, Lu Yan, Xiaolong Jin, Shengwei An, Shiqing Ma, and Xiangyu Zhang. Bait: Large language model backdoor scanning by inverting attack target. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 1676–1694. IEEE Computer Society, 2025. doi: 10.1109/SP61157.2025.00103. URL https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00103.

[22] Brandon Tran, Jerry Li, and Aleksander Madry. Spectral signatures in backdoor attacks, 2018. URL https://arxiv.org/abs/1811.00636.

[23] Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y. Zhao. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 707–723. IEEE, 2019. doi: 10.1109/SP.2019.00031. URL https://ieeexplore.ieee.org/document/8835365.

[24] Yifei Wang, Dizhan Xue, Shengjie Zhang, and Shengsheng Qian. Badagent: Inserting and activating backdoor attacks in llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9811–9827. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.530. URL https://aclanthology.org/2024.acl-long.530/.

[25] Zihan Wang, Rui Zhang, Hongwei Li, Wenshu Fan, Wenbo Jiang, Qingchuan Zhao, and Guowen Xu. Confguard: A simple and effective backdoor detection for large language models, 2025. URL https://arxiv.org/abs/2508.01365.

[26] Wenkai Yang, Xiaohan Bi, Yankai Lin, Sishuo Chen, Jie Zhou, and Xu Sun. Watch out for your agents! investigating backdoor threats to llm-based agents, 2024. URL https://arxiv.org/abs/2402.11208.

[27] Biao Yi, Tiansheng Huang, Sishuo Chen, Tong Li, Zheli Liu, Zhixuan Chu, and Yiming Li. Probe before you talk: Towards black-box defense against backdoor unalignment for large language models, 2025. URL https://arxiv.org/abs/2506.16447.

[28] Shuai Zhao, Meihuizi Jia, Zhongliang Guo, Leilei Gan, Xiaoyu Xu, Xiaobao Wu, Jie Fu, Yichao Feng, Fengjun Pan, and Luu Anh Tuan. A survey of recent backdoor attacks and defenses in large language models, 2024. URL https://arxiv.org/abs/2406.06852.

[29] Mengxin Zheng, Jiaqi Xue, Xun Chen, Yanshan Wang, Qian Lou, and Lei Jiang. Trojfsp: Trojan insertion in few-shot prompt tuning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1141–1151. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.naacl-long.64. URL https://aclanthology.org/2024.naacl-long.64/.